

```
// 5 loop types  
// real examples  
// mini project
```

— MODULE 5

Loops

Repeating Tasks in Websites

```
for (let i = 0; i < Infinity; i++) { learnJS(); }
```

— JavaScript Course

THE PROBLEM

Why Do We Need Loops?

! Without Loops

```
displayProduct(product1);  
displayProduct(product2);  
displayProduct(product3);  
displayProduct(product4);  
displayProduct(product5);  
...  
displayProduct(product500);
```

500 lines of repetitive code.
Inefficient and hard to maintain.

✓ With a Loop

```
for (let i = 0; i < products.length; i++) {  
    displayProduct(products[i]);  
}
```

3 lines. Works for 5, 500, or 5,000 products.
Easy to maintain and scale.

| Loops **automate repetition** — the same code runs for any number of items.

WHAT YOU'LL LEARN

Topics Covered

01 What is a Loop?

Definition, flow of execution & infinite loops

03 Loop Control

break & continue statements

05 Infinite Loops

Why they happen & how to avoid them

02 Types of Loops

for, while, do...while, for...of, for...in

04 Nested Loops

Loops inside loops, use cases

06 Performance

Efficiency considerations & best practices

+ real website examples & mini project

LOOP TYPE 1

The **for** Loop

// Syntax

```
for (initialization; condition; increment) {  
    // code to repeat  
}
```

Initialization

Start value — runs once

Condition

Keep running while true

Increment

Update each iteration

// Example — display 500 products

```
for (let i = 0; i < products.length; i++) {  
    displayProduct(products[i]);  
}
```

WEBSITE USES

Display Products

Loop through an e-commerce catalog

Show Notifications

Render a list of alerts

Generate Table Rows

while & do...while Loops

while – Entry Controlled

```
while (condition) {  
    // code runs here  
}
```

Checks condition **before** each iteration.

// Website Uses:

- Retry login attempts
- Wait for user input
- Loading / polling data

do...while – Exit Controlled

```
do {  
    // runs at least once  
} while (condition);
```

Runs **at least once**, then checks.

// Website Uses:

- Show popup at least once
- Ask for confirmation
- Form retry on failure

| while = check first. do...while = run first, then check.

for...of & for...in Loops

for...of – Iterates Values

```
for (let product of products) {  
  display(product);  
}
```

Works on **arrays, strings & collections**

// Website Uses:

- Display products
- Show categories
- Render reviews

for...in – Iterates Keys

```
for (let key in userProfile) {  
  show(key, userProfile[key]);  
}
```

Works on **object properties**

// Website Uses:

- User profile display
- Settings panels
- API response objects

| for...of = array values. for...in = object keys.

break & continue

| break

Exits the loop **immediately**. No more iterations run.

```
for (let product of products) {  
  if (product.id === targetId) {  
    found = product;  
    break; // stop searching!  
  }  
}
```

// Website Example:
Stop searching when a product is found.

| continue

Skips the **current iteration**. Loop keeps going.

```
for (let product of products) {  
  if (!product.available) {  
    continue; // skip this one  
  }  
  displayProduct(product);  
}
```

// Website Example:
Ignore unavailable products.

Nested Loops

```
// A loop inside a loop
for (let row = 0; row < rows; row++) {
  for (let col = 0; col < cols; col++) {
    renderCell(row, col);
  }
}
```

// How it works

The outer loop runs once. For each outer iteration, the inner loop runs **completely** from start to finish.

WEBSITE USES

Seat Booking

Outer loop = rows, inner loop = seat positions

Calendar Generation

Outer loop = weeks, inner loop = days

Product Grid

Outer loop = categories, inner loop = items

Keep nesting minimal on large datasets — performance degrades quickly with deeply nested loops.

WATCH OUT!

Infinite Loops & Performance

INFINITE LOOPS

```
// This never ends – browser freezes!  
  
let i = 0;  
while (i < 10) {  
  console.log(i);  
  // forgot i++ !  
}
```

Common Causes

- Forgetting to increment the counter
- Condition that never becomes false
- Modifying the wrong variable

PERFORMANCE TIPS

❗ Avoid Unnecessary Loops

Don't loop when you don't need to

❗ Use the Right Loop

Match the loop type to the task

❗ Minimize Nesting

Especially on large datasets

❗ Keep Loop Bodies Simple

Simpler code runs faster and is easier to debug

LOOPS IN ACTION

Real Website Examples



E-Commerce

// Loop through:

- Products catalog
- Cart items
- Price calculations



Social Media

// Display:

- Posts & comments
- Notifications
- Friend lists



Dashboard

// Generate:

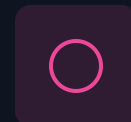
- Statistics & reports
- Charts
- Activity feeds



Student Portal

// Loop through:

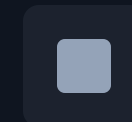
- Students & subjects
- Marks & grades
- Timetables



Shopping Cart

// Calculate:

- Total quantity & price
- Tax & discount
- Shipping costs



Admin Panel

// Display:

- Users & orders
- Messages
- Payments

AVOID THESE

Common Mistakes

01 Infinite Loops

Forgetting to update the loop variable — the browser freezes

02 Incorrect Loop Conditions

Using `<` vs `≤` incorrectly, leading to skipped or extra iterations

03 Off-by-One Errors

Starting at 1 instead of 0, or looping one too many or too few times

04 Modifying the Loop Variable Incorrectly

Changing `i` inside the loop body when it should only update in the increment

05 Using the Wrong Loop Type

Using `for ... in` on arrays instead of `for ... of`

DO THIS INSTEAD

Best Practices

Use **for** when you know the number of iterations

Perfect for arrays with a known length

Use **for ... of** for arrays

Cleaner syntax, no index management needed

Use **for ... in** for objects

Designed for iterating over object keys

Keep loop bodies simple

Extract complex logic into separate functions for clarity

Avoid deeply nested loops when possible

Refactor with helper functions to reduce complexity

Mini Project

// Build this

Product Listing Page

A simple web page where JavaScript loops through an array of products and displays them dynamically in HTML.

Why This Project?

- Connects loops to real DOM manipulation
- Simulates a real e-commerce use case
- Prepares you for Module 6: Functions

// Starter code structure

```
const products = [
  { name: "Laptop", price: 999 },
  { name: "Phone", price: 599 },
  { name: "Tablet", price: 399 },
];

for (let product of products) {
  const card = document.createElement("div");
  card.innerHTML = `
    <h3>${product.name}</h3>
    <p>${product.price}</p>
  `;
  container.appendChild(card);
}
```


Learning Outcomes

By the end of this module, you can:

- Understand why loops are essential
- Use all major JavaScript loop types
- Choose the right loop for different scenarios
- Apply loops to arrays and objects
- Avoid common looping mistakes
- Build dynamic website features using repetition

— UP NEXT

Module 6

JavaScript Functions

Organize code into reusable blocks. One of the most important JavaScript topics — used in every interactive feature on modern websites.

Functions power:

- Login handling
- API calls
- Form validation
- Button actions